Zephyr Project

IT-Schulungen.com
Technology & Transformation

Praxis 2: Starten und Beobachten

Live-Demonstration und Einzelarbeit

1.08

Vom Build zum beobachteten Prozess

Fünf Schritte mit sichtbarem Laufzeitergebnis

01

Starten

west run und direkter Programmstart

02

Lesen

Boot-Banner, Version und Anwendungs-ausgabe erkennen

03

Steuern

Hilfe aufrufen und Prozess sicher beenden

04

Verändern

Quelltext anpassen und inkrementell neu bauen

05

Prüfen

Ausgabe, Prozess-status und Zeit-verhalten erklären

Ergebnis: Das Programm lässt sich auf beiden Wegen starten, beobachten und zuverlässig beenden.

Zwei Startwege gezielt vergleichen

BUILD-ZIEL

west build -t run

nutzt den aktuellen Build
startet das konfigurierte Ziel
praktisch im Entwicklungszyklus

DIREKTSTART

zephyr.exe

Linux-Prozess direkt ausführen
Pfad und Rechte sichtbar machen
Exit-Status separat prüfen

BEOBACHTUNG

Gleiches Programm

identische Anwendungsausgabe
gleiche Konfiguration
unterschiedlicher Startweg

Beide Wege verwenden dasselbe Build-Ergebnis – der Unterschied liegt im Aufruf.

Programm starten und Optionen prüfen

TERMINAL

```
$ west build -t run
$ ./build/zephyr/zephyr.exe
$ ./build/zephyr/zephyr.exe --help
```

01 Startweg 1

west ruft das passende Run-Ziel des vorhandenen Builds auf.

02 Startweg 2

Der direkte Aufruf zeigt: zephyr.exe ist ein ausführbarer Linux-Prozess.

Zuerst die Ausgabe beobachten; anschließend mit --help die verfügbaren Laufzeitparameter erkunden.

Boot- und Anwendungsausgabe lesen

KONSOLENAUSGABE

```
*** Booting Zephyr OS build v… ***
Hello World! native_sim
```

01 Boot-Banner

Zeigt den Start des Zephyr-Systems und enthält Build-Informationen.

02 Versionsangabe

Ordnet die Ausgabe dem verwendeten Zephyr-Stand zu.

03 Anwendungsausgabe

Stammt aus main() und bestätigt den ausgeführten Quelltext.

Markiert Banner, Version und Anwendungstext – und nennt jeweils deren Ursprung.

Prozesszustände einordnen

`Ctrl+C`	Unterbricht den Hostprozess; der Rückgabestatus zeigt die Beendigung durch ein Signal
`main() kehrt zurück`	Der Kernel läuft grundsätzlich weiter und kann in den Idle-Zustand wechseln
`Idle-Zustand`	Keine sichtbare Ausgabe bedeutet nicht automatisch, dass der Prozess beendet ist
`Kontrolliertes Testende`	Ein vorgesehener Exit-Pfad liefert einen auswertbaren Prozessstatus
`Beobachtungsregel`	Ausgabe, laufenden Prozess und Exit-Status getrennt prüfen

Nicht nur auf die letzte Textzeile schauen: Der Linux-Prozess kann danach weiterhin aktiv sein.

Simulationszeit und Host-Effekte

BEOBACHTEN STATT ANNEHMEN

native_sim führt Zephyr als Hostprozess aus – Zeit und Prozessumgebung bleiben wichtige Messgrößen.

Simulierte Zeit	kann unabhängig von der real verstrichenen Host-Zeit fortschreiten
Schnelle Ausführung	beschleunigt Tests und kann Ausgaben sehr dicht erscheinen lassen
Determinismus	gleiche Eingaben erleichtern reproduzierbare Beobachtungen
Host-Einflüsse	Scheduling, Pufferung und Signale gehören zur Linux-Seite
Grenzen	MCU-Timing und konkrete Peripherie werden damit nicht bewiesen

Für Testergebnisse immer unterscheiden: Zephyr-Verhalten, Simulationsmodell und Host-Effekt.

Ändern, bauen, prüfen

TERMINAL

```
$ ${EDITOR:-nano} samples/hello_world/src/main.c
$ west build
$ west build -t run

# nach dem Beenden
$ echo $?
```

01 Inkrementeller Build

Ohne -p always werden nur betroffene Schritte erneut ausgeführt.

02 Laufzeitprüfung

Die neue Meldung und der Prozessstatus belegen, dass die Änderung aktiv ist.

Änderung, Build-Log und neue Ausgabe gemeinsam dokumentieren – nicht nur das Endergebnis.

Praxis 2 abgeschlossen

ERFOLG

Programm beherrscht

beide Startwege genutzt

Ausgabe zugeordnet

Hilfe untersucht

Prozess sicher beendet

PARTNERCHECK

Beobachtung erklären

Idle und Ende unterscheiden

Zeitverhalten beschreiben

Änderung nachweisen

Exit-Status einordnen

NÄCHSTER SCHRITT

Praxis 1.09

Anwendungsstruktur lesen

CMakeLists.txt einordnen

prj.conf verstehen

main() als Thread betrachten

Lernziel: Starten, Beobachten, Verändern und zuverlässig Beenden.